

Workflow, Not Prose: A Multi-Agent Methodology for Data Agents

Chia-liang Kao
Recce
United States

Kent Huang
Recce
Taiwan

1 Abstract

LLM agents on the Data Agent Benchmark (DAB) [1] fail on roughly half of queries, with planning and implementation faults dominating the failure-mode breakdown. We present two methodology contributions: (i) a **declarative-markdown harness** whose *mutation surface*, the file-set changed between two otherwise-identical runs, is small by construction (the multi-agent architecture is itself declarative: stage boundaries, agent roles, and inter-agent dispatch live in the workflow rather than in agent code or megaprompts), making single-mutation comparisons attributable rather than confounded with harness changes; and (ii) an **auto-research loop** in which failure-mode analysis on completed experiments proposes new hypotheses, optionally human-reviewed, that progress through the same experiment workflow. **A pre-specified ablation on Claude Opus 4.6 (N=5, hints OFF) scores 0.572 pass@1 on the full 12-dataset DAB benchmark. The prose-methodology configuration is null (-0.008); the entire architecture-effect gain is attributable to the agent-boundary + fresh-context-verify mutation (+0.036).** The architecture lift concentrates on the 8 datasets the auto-research loop iterated on and does not transfer to the 4 un-iterated datasets, a generalization gap the small-mutation harness exposes.

2 Introduction

Data agents on DAB fail on the majority of queries: the DAB-published Opus 4.6 ReAct baseline reaches 0.4376 pass@1 [1].¹ A 50-trial pass@50 ceiling of roughly 69% leaves a third of the benchmark unsolved by any sample. The DAB paper attributes that gap to three failure-mode (FM) classes: ~40% planning errors (FM-plan), ~45% implementation errors (FM-impl), and ~15% data-discovery errors (FM-data) [1]. Existing agent-side responses fall into two camps. ReAct-style think-act loops [2] add intermediate reasoning steps inside a single trajectory, and sampling-based methods (best-of-N, self-consistency [3], self-refinement [4, 5]) trade compute for variance reduction across trajectories. Both treat reliability as a property of the agent’s stochastic decoding rather than of the experimental setup that produces the trajectory.

We show that a declarative-markdown harness, in which the solver’s stages and the experiment’s configuration live as YAML-frontmatter state machines, each stage paired with a brief directive and an explicit output checklist (Figure~1), makes single-mutation comparisons attributable. A

pre-specified ablation isolates the lift to a structural mutation, agent-boundary plus fresh-context verification, rather than to the prose methodology that ostensibly motivates it. The lift concentrates on the eight datasets the auto-research loop iterated on and does not transfer to the four un-iterated, a generalization gap the same small-mutation discipline exposes.

3 Method

The harness declares its solver as plain-text frontmatter dispatched by a markdown workflow runtime, so an experiment is fully described by a small text diff. We use *mutation surface*, the set of files an experimenter changes between two otherwise-identical runs, as the lens that ties stage design to comparability across configurations.

3.1 Inner workflow

The solver replaces a single ReAct loop [2] with three role-restricted stages declared in the YAML header of the workspace README: **modeling** enumerates tables, samples values, identifies join keys, and writes a context document; **analysis** reads that document, answers every question, and writes an answer file plus a reasoning trace; **verification** re-derives each answer from the same context document without seeing the analysis stage’s narrative, then either passes or rejects with numbered findings. Each stage runs as a fresh agent whose only inputs are the artefacts of the prior stage, so the stage boundary doubles as a context reset. Rejection routes back to **analysis** through a bounded feedback edge, at most three cycles, so a fresh adversary can challenge the original plan without anchoring on it. The prose below the YAML header acts as the per-stage prompt the dispatched agent reads. These three stages were not designed top-down; they emerged from a sequence of pre-specified hypotheses (single-agent prose methodology, DuckDB-ATTACH unification, batching, SQL-first execution, multi-stage scaffolding), each tested as a separate hypothesis through the outer workflow before the architecture above was committed (see §6).

Figure~1 shows both scales of state machine side-by-side. Each candidate configuration is tracked as a *hypothesis entity*: a single markdown file whose YAML frontmatter encodes its current state and progression through the outer workflow. The next section describes that scaffolding.

3.2 Mutation surface

Every harness mutation is either a frontmatter edit or a small text diff against **main**. The hypothesis entity’s frontmatter

*Code and experiments: <https://github.com/clkao/dataagentbench>

¹All scores reported in this paper are DAB’s stratified pass@1, per-dataset pass rate first, then unweighted mean across the 12 datasets, to match the leaderboard convention. See §3.

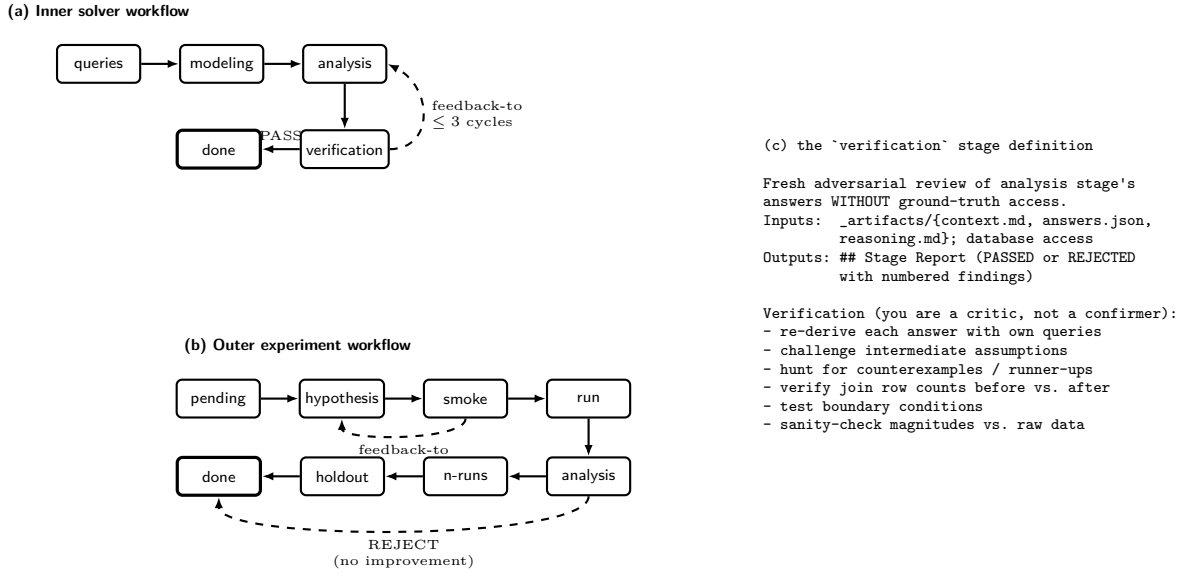


Figure 1: The harness has two state machines, both declared as YAML in tracked markdown files and dispatched by the same runtime. (a) the *inner solver workflow* (per query) runs three role-restricted stages with a bounded verification→analysis feedback edge (≤ 3 cycles). (b) the *outer experiment workflow* (per experiment) tracks each candidate configuration through eight states from pending to done. (c) example stage definition for verification: the YAML header plus this prose body is the dispatched agent’s prompt. Together (a) and (c) define the experimenter’s mutation surface; (b) is the experiment-tracking layer.

declares the agent-affecting variables in an allow-listed set of fields: underlying model (e.g., `claude-opus-4-6`), effort level, coding-agent harness (Claude Code, Codex CLI, etc.), available tools, enabled skills (versioned plugin pins), query mode (*fresh* vs. *batch*), and hints flag. Because stage definitions, prompt templates, and plugin pins all live in markdown files, an experiment that changes the solver appears as a small, reviewable text diff, and the mutation surface shrinks to two well-known files: the workspace README for the inner workflow and the hypothesis entity itself.

4 Experimental Setup

We evaluate on DataAgentBench (DAB) [1], which contains 12 datasets and 54 queries spanning four DBMSes, MongoDB, PostgreSQL, SQLite, and DuckDB. The benchmark itself ships all 54 queries as one evaluation set; the leaderboard convention is the full-12 pass@1 reported in Table-2. For our internal auto-research loop we additionally separate eight *iterated* datasets (37 queries) the harness team learned from when proposing FM-targeted mutations from four *un-iterated* datasets (17 queries) whose failures the loop is forbidden to read. This split is ours, not DAB’s; we use it only to expose generalization gaps in §5.4. All scores are DAB’s stratified pass@1, with every dataset weighted equally regardless of query count.

We compare three configurations that share the workspace bootstrap, the DAB query files, the model (Claude Opus 4.6 at low effort), the DuckDB-ATTACH unified query layer, and `query_mode: fresh` (each query runs in a clean session with

no within-dataset context carryover); they differ only in the `workspace/README.md` the agent reads and in the dispatch policy.

Table 1: Pre-registered ablation. Each row is a configuration; the mutation surface between adjacent rows is one workspace-README diff plus the dispatch policy.

Configuration	README contents	Dispatch
<code>direct-minimal</code>	file orientation + DB rules	1 agent
<code>direct-structured</code>	+ methodology prose	1 agent
<code>spacedock</code>	+ YAML workflow schema	first-officer + 3 ensigns

`direct-minimal` is Claude Code with no plugins, no skills, and no methodology prose; its README documents only the workspace file layout and DuckDB attach rules. The `spacedock` configuration’s dispatch uses the harness’s runtime: a dispatcher skill/agent (*first-officer*) that spawns one fresh subagent (*ensign*) per stage; `spacedock` is the runtime’s codename, distributed as Claude Code and Codex CLI plugins. The other two configurations use the same runtime invoked as a single agent.

Hints are OFF in all three configurations; the DAB-published Opus 4.6 ReAct baseline runs with `-use_hints ON`. The hints-matched comparison is committed for camera-ready (§5).

We run $N=5$ per configuration across the full 12 datasets. The architecture-effect comparison (`direct-structured` → `spacedock`) is the central comparison; the prose-methodology

comparison (direct-minimal $\rightarrow$ direct-structured) is the disconfirmation control. Each configuration is an entity that moves through the outer 8-state workflow (pending $\rightarrow$ hypothesis $\rightarrow$ smoke $\rightarrow$ run $\rightarrow$ analysis $\rightarrow$ n-runs $\rightarrow$ holdout $\rightarrow$ done), with each transition recorded as a git commit on the entity file.

5 Results

5.1 Scoreboard

The full-12 scoreboard places spacedock at 0.572. We report the DAB-published Opus 4.6 ReAct baseline (0.4376) as a context anchor; direct cross-system comparison with other leaderboard entries is out of scope for this report because they were scored under pre-PR-#41 validators.

Table 2: Pass@1 on the full 12-dataset DAB benchmark (54 queries). Our three configurations (lower block, hints OFF, Claude Code coding-agent harness). σ is the standard deviation of per-run scores across N=5 runs.

System (Opus 4.6 unless noted)	Pass@1	σ
DAB ReAct [1]	0.4376	n/a
direct-minimal	0.544	0.010
direct-structured	0.536	0.044
spacedock	0.572	0.059

5.2 Causal decomposition

The direct-minimal $\rightarrow$ direct-structured prose-methodology effect across the full 12 datasets is -0.008 . The direct-structured $\rightarrow$ spacedock architecture effect is 0.036 . The total direct-minimal $\rightarrow$ spacedock swing is 0.028 . At N=5 with $\sigma_{\text{spacedock}} = 0.059$, this is a nominal lift of roughly 1.1 pooled standard errors; we report it as such pending the per-query McNemar test committed for camera-ready (§6c). The mutation surface for direct-minimal $\rightarrow$ direct-structured is a single README append (the methodology prose); for direct-structured $\rightarrow$ spacedock it is a coupled *structural envelope*: the YAML workflow schema in the README (declaring the stages and the rejection edge) and the multi-agent dispatch policy that consumes it, treated as one mutation because neither half does work alone. The added README content is interface metadata only the first-officer acts on; a single agent reading it would ignore the schema, so the 0.036 lift is attributable to the dispatch behavior the schema enables rather than to README prose alone.

5.3 Mechanism: fresh-context verify

On bookreview/q1, a “which decade has the highest average rating” question with ground truth “2020”, direct-minimal and direct-structured both hallucinated **1980s** despite direct-structured’s prose telling the single agent to adversarially review its own answer. Spacedock’s verification ensign, reading the answer file and reasoning trace in fresh context without the analysis ensign’s narrative, independently re-derived

2020s (correct). The trace recorded 57 tool-use blocks and three subagent dispatches (modeling / analysis / verification). This is one query, but it matches the predicted mechanism: prose-instructed self-review cannot substitute for a fresh agent, because the single agent retains anchoring bias on its own prior reasoning.

5.4 Failure-mode attribution

Per-query analysis surfaces two dataset-level patterns. First, **PATENTS is a 0/3 stable failure** for every Opus configuration we have run, including spacedock on the un-iterated subset: the queries require regex-defeating CPC-classification text extraction, exactly the FM-impl class the DAB paper highlights. Second, **crmarenapro identifier corruption is the dominant FM-data surface**: the paper documents reformatted IDs and trailing-whitespace identifiers across 26 of 54 queries [1], and spacedock’s -0.06 against DAB’s hinted ReAct on this dataset is consistent with hint-conditioned identifier canonicalisation that we run without. Each pattern names one mutation a future entity can target: LLM-based Named Entity Recognition (NER) on PATENTS, identifier-canonicalisation on crmarenapro, each a single new markdown entity moving through the same outer state machine.

6 Discussion

The clean finding is the asymmetry inside the decomposition: prose methodology alone moves nothing (-0.008 across the full 12 datasets), while the agent-boundary + fresh-context-verify architecture moves 0.036 . For Opus 4.6 at low effort, workflow scaffolding, not prose scaffolding, produces the lift; the Haiku datapoint below shows the prose/workflow ratio is capability-dependent, so the title is a hypothesis the data support for this configuration, not a universal claim. Because the mutation surface for each comparison is one README append plus the dispatch policy, the asymmetry is attributable to the structural intervention rather than to confounded harness changes.

The bookreview/q1 vignette shows why prose-instructed self-review cannot substitute for a fresh agent: a single agent “verifying its own work” anchors on its prior reasoning and tends to confirm what it already decided. A new agent that re-derives from scratch breaks the anchor, and that mechanism cannot be simulated by prompt instructions alone, the single agent still has anchoring bias.

Iteration leakage check. Our auto-research loop is permitted to read failure traces from only 8 of the 12 datasets; the other 4 (yelp, PANCANCER_ATLAS, GITHUB_REPOS, PATENTS) are held aside and the loop is forbidden to look at their per-query failures. The architecture effect concentrates entirely on the 8 iterated datasets (spacedock 0.647 , direct-minimal 0.591 , $+0.056$) and reverses on the 4 un-iterated ones (spacedock 0.422 , direct-minimal 0.451 , -0.029). We report this rather than hide it: the methodology contribution does not promise generalization; it promises that when generalization fails — even when our own iteration is the cause — the failure is attributable. A properly-blinded architectural-value

claim would freeze the workflow and run against a separately-held second holdout (committed for camera-ready).

A second, smaller datapoint orthogonal to the Opus ablation: on Claude Haiku 4.5, a one-sentence SQL-first constraint appended to the analysis stage’s workflow definition lifts dev-set naive scoring by +14pp (35% $\rightarrow$ 49%) while regressing Opus on the same mutation by three queries on agnews [1]. The constraint helps the weaker model by forcing schema-first exploration but constrains Opus’s already-effective free-form workflow. Small-mutation discipline makes the asymmetric sign of one line attributable across capability gradients.

Wenz et al.~argue that data-agent reliability comes from constraining the LLM’s query surface at the data layer [6]; we constrain the experimenter’s mutation surface at the harness layer, independent constructions that may compose, since shrinking the surface where surprise enters is the shared move. The declarative-vs-imperative axis distinguishes our markdown-diff harness from imperative-search prior work: ADAS [7] and AFlow [8] search over Python-coded agentic workflows; Meta-Harness [9] optimizes full Python harness programs end-to-end via an LLM proposer; DSPy [10] compiles declarative LLM programs to imperative pipelines. Our harness keeps the optimization surface as plain-text markdown diffs, so every variant is a reviewable, mergeable git commit rather than a generated Python program, yet the diff still represents a multi-agent topology and role-specific checklists.

This loop is the auto-research pattern [11] in miniature, scoped to harness-level mutations rather than training-loop kernels. Failure-mode analysis on a completed entity’s traces produces structured findings (§4.5); each finding admits one or more candidate hypotheses, expressed as new entity files moving through the same workflow. Hypotheses run through the loop included: *does prose-only modeling/analysis/verification methodology help a single agent?*; *does unifying multi-DBMS access through DuckDB ATTACH improve cross-database queries?*; *does forcing SQL-first execution prevent Python-loop antipatterns on weaker models?*; *does multi-stage scaffolding compound with higher reasoning effort?* The architecture of §3.1 is the union of what survived. The loop also ate its own definition: the experiment workflow’s preflight, smoke, and n-runs stages were each added as workflow-improvement entities, after the corresponding failure modes (silent infrastructure fallback, runaway full-runs, bimodal score distributions) were attributed in earlier rounds.

Camera-ready commitments. Three deliverables are pre-specified for the camera-ready version: (a) a hints-matched spacedock + `-use_hints` run; (b) a second-held un-iterated subset with the inner-solver workflow frozen pre-iteration, to disentangle the iteration-leakage confound from the architectural lift; (c) per-query analysis of the architecture effect, since N=5 aggregate comparisons leave the lift’s significance uncertain. Each is a single markdown entity moving through the same outer state machine; we archive negative results alongside positive ones.

References

- [1] Xinyue Ma et al. Can ai agents answer your data questions? a benchmark for data agents. 3 2025. URL <https://arxiv.org/abs/2603.20576>. arXiv preprint arXiv:2603.20576.
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023. arXiv preprint arXiv:2210.03629.
- [3] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.
- [4] Aman Madaan et al. Self-refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.
- [5] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- [6] Fabian Wenz, Felix Treutwein, Kai Arenja, Çağatay Demiralp, and Michael Stonebraker. An alternate agentic ai architecture (it’s about the data). 4 2026. URL <https://arxiv.org/abs/2604.21413>. arXiv preprint arXiv:2604.21413.
- [7] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *ICLR*, 2025.
- [8] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. In *ICLR*, 2025. Oral presentation.
- [9] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. 3 2026. URL <https://arxiv.org/abs/2603.28052>.
- [10] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In *ICLR*, 2024.
- [11] Andrej Karpathy. AutoResearch: Ai agents running research on single-gpu nanochat training automatically. <https://github.com/karpathy/autoresearch>, 3 2026. GitHub repository.